

Test Plan QA Pro REST App:

Test Performed:

- [Stress Test](#)
- [Load Test](#)

Test Plan consists of the following Config Elemets:

- **HTTP Request Defaults** (is used to set common server, port, and protocol values for all HTTP requests, which simplified test configuration and reduced repetition)
 - Protocol: http
 - Server Name or IP: local host
 - Port Number: 3001

HTTP Request Defaults

Name:

Comments:

☒ Basic ☐ Advanced

Web Server

Protocol [http]: Server Name or IP: Port Number:

HTTP Request

Path: Content encoding:

☒ Parameters ☐ Body Data

☐ Send Parameters With the Request

- **HTTP Header Manager** (is used to set Content-Type header for all HTTP requests, which is especially important for POST and PUT requests.)
 - Content-Type: application/json

HTTP Header Manager

Name:

Comments:

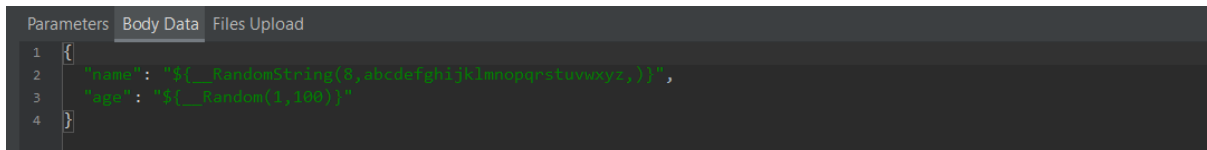
Headers Stored in the Header Manager

Name	Value
Content-Type	application/json

Jmeter Test Plan consists of the following requests, which are located in the fragment:

GetCharacters	GET
CreateCharacter	POST
GetCharacterById	GET
UpdateCharacter	PUT
DeleteAddedUser	DELETE

The body of the **CreateCharacter POST** request is structured as follows to generate random user data:

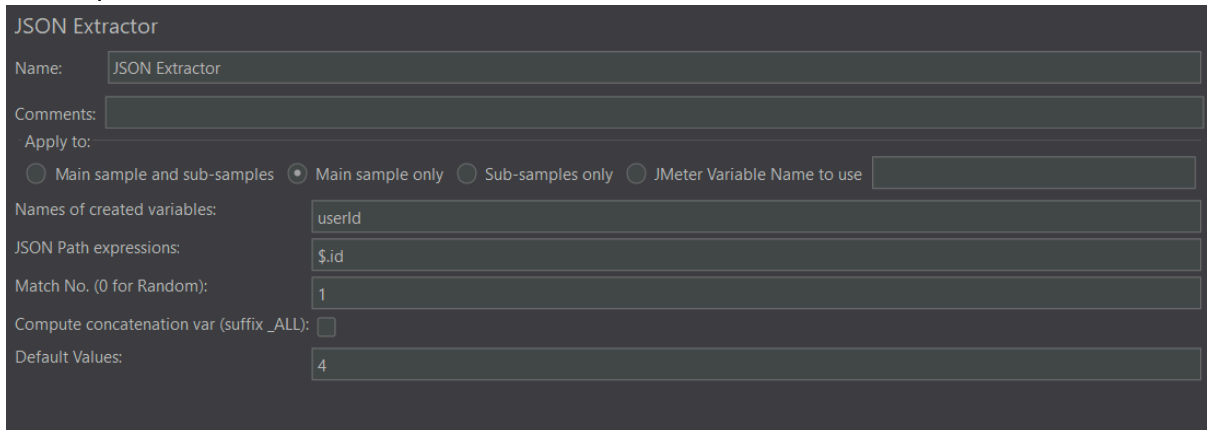


The screenshot shows the 'Body Data' tab of a JMeter request configuration. It contains a JSON body with two fields: 'name' and 'age'. The 'name' field is assigned a random string of 8 characters from the set 'abcdefghijklmnopqrstuvwxyz', and the 'age' field is assigned a random integer between 1 and 100.

```
1 {  
2   "name": "${__RandomString(8,abcdefghijklmnopqrstuvwxyz,1)}",  
3   "age": "${__Random(1,100)}"  
4 }
```

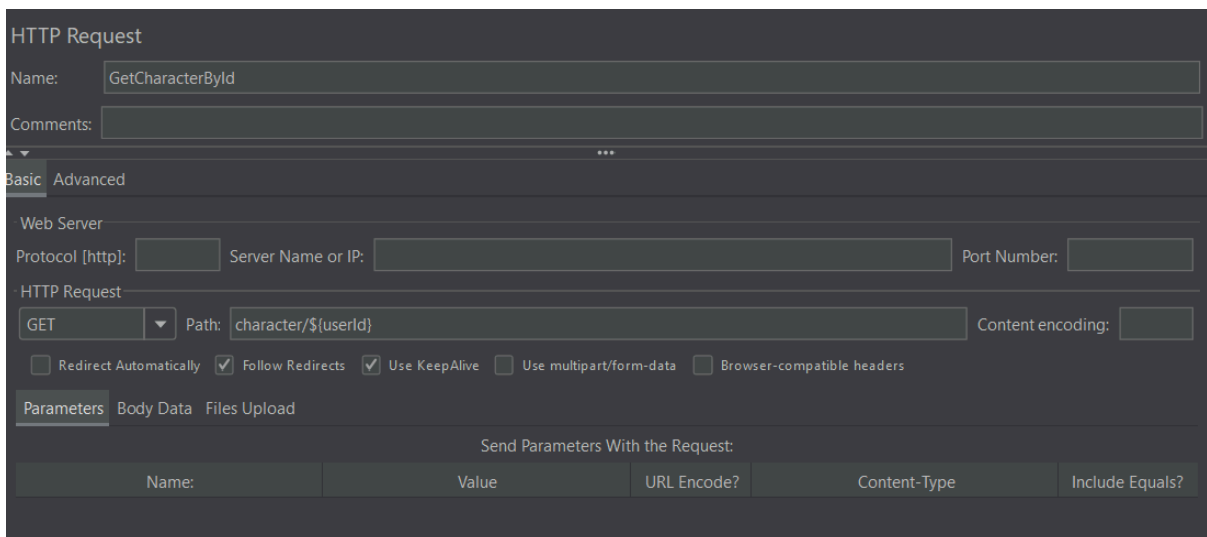
The same structure is used for **UpdateCharacter PUT** to randomly update character's data, also.

The request includes a JSON Extractor to retrieve the ID:



The screenshot shows the 'JSON Extractor' configuration in JMeter. The 'Name' field is 'JSON Extractor'. The 'Apply to' section has 'Main sample only' selected. The 'Names of created variables' field contains 'userId'. The 'JSON Path expressions' field contains '\$.id'. The 'Match No. (0 for Random)' field contains '1'. The 'Default Values' field contains '4'.

Subsequent **GetCharacterById GET**, **UpdateCharacter PUT**, **DeleteAddedUser DELETE** requests use `userId` variable obtained from previous request:



The screenshot shows the 'HTTP Request' configuration in JMeter. The 'Name' field is 'GetCharacterById'. The 'Basic' tab is selected. The 'Web Server' section shows 'Protocol [http]', 'Server Name or IP', and 'Port Number'. The 'HTTP Request' section shows 'GET' as the method and 'character/\${userId}' as the path. The 'Parameters' tab is also visible at the bottom.

Listeners:

- **Synthesis Report**
used to provide an overall summary of the test results, including key performance metrics like average response time, throughput, and error percentage
- **Active Threds Over Time**
used to visualize the number of virtual users (threads) active during the test, helping to understand load patterns

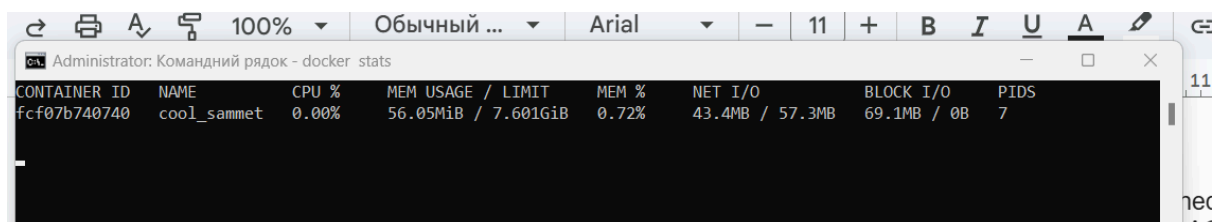
- **Response Latencies Over Time**
used to track how request response times changed during the test, helping to spot slowdowns or spikes.
- **Transactions per Second**
used to measure the throughput of requests processed by the system, showing how many transactions were completed per second.

!!PerfMon ServerAgent v2.2.3 was attempted, however it crashed due to native SIGAR library failure on Windows 11 with OpenJDK 11 (EXCEPTION_ACCESS_VIOLATION). Therefore Docker statistics (**docker stats**) and Windows Task Manager were used to monitor CPU and memory utilization during the load tests.

The tests were run in a Docker container (4.58.0), and baseline metrics from **docker stats** below:

Before Load:

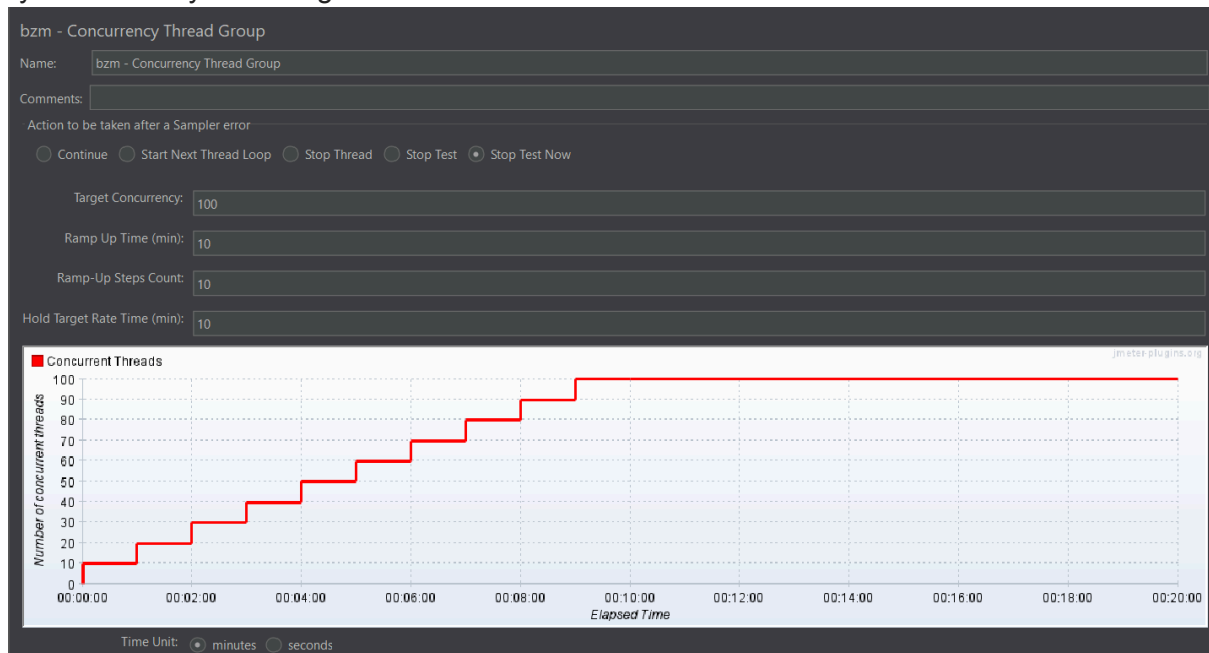
- CPU: 0.00% – almost no CPU usage.
- Memory: 56 MiB / 7.6 GiB (0.72%) – very low, plenty of free memory.
- Network I/O: 43.4 MB / 57.3 MB – low network activity.
- Block I/O: 69.1 MB / 0B – minimal disk usage.
- PIDs: 7 – few running processes.



CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
fcf07b740740	cool_sammet	0.00%	56.05MiB / 7.601GiB	0.72%	43.4MB / 57.3MB	69.1MB / 0B	7

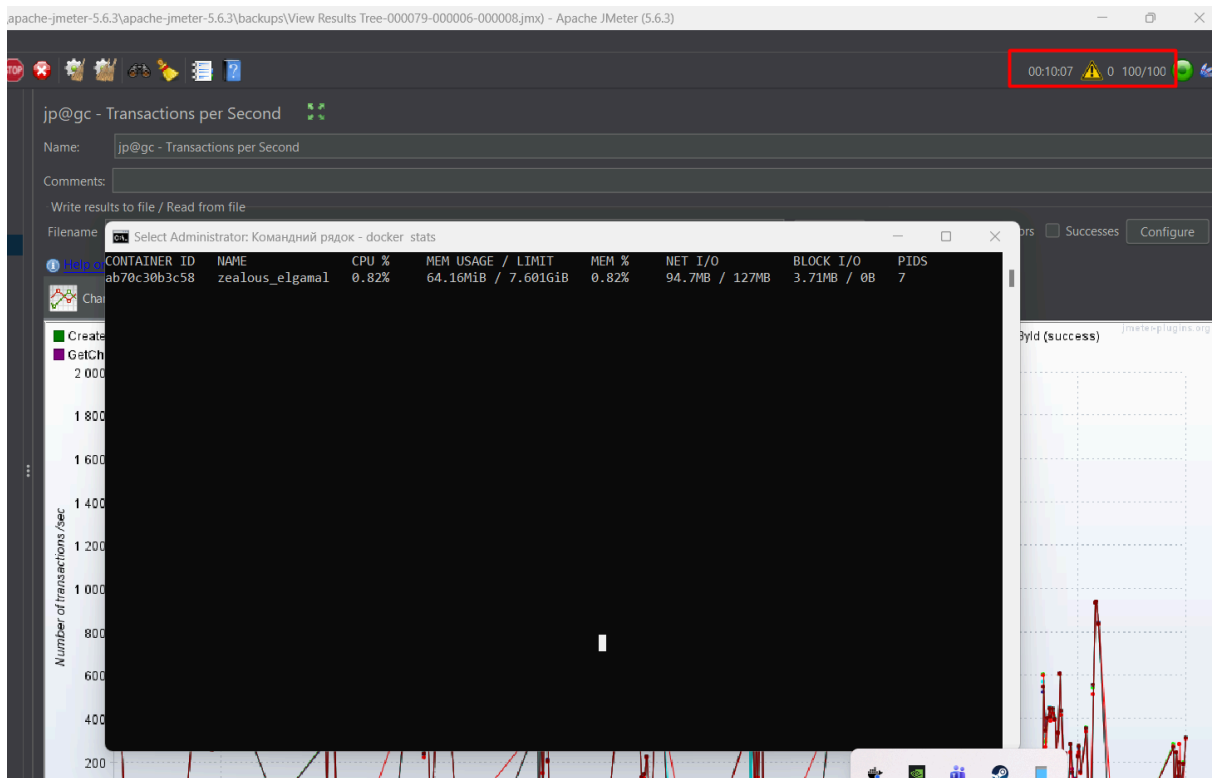
Stress Test Configuration

A Concurrency Thread Group was configured with a target of 100 concurrent users, ramp-up time of 10 minutes with 10 incremental steps, and a hold duration of 10 minutes to evaluate system stability under high load.



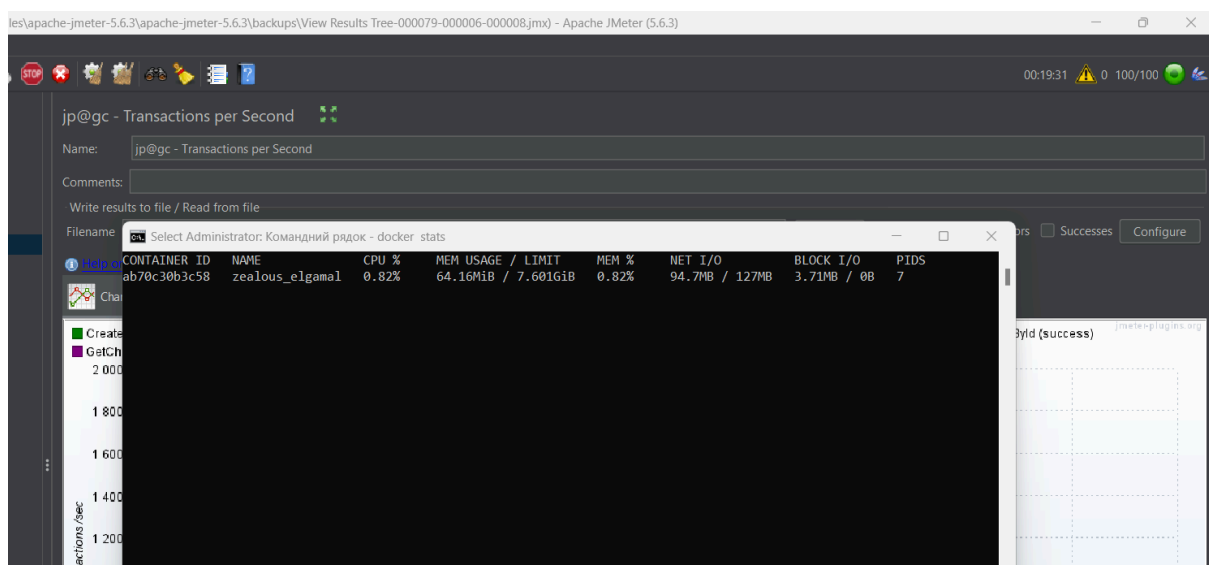
Peak Load:

- CPU: 0.82% – still very low, CPU not stressed.
- Memory: 64.16 MiB / 7.6 GiB (0.82%) – slightly higher but still very low.
- Network I/O: 94.7 MB / 127 MB – traffic increased roughly 2x.
- Block I/O: 3.71 MB / 0B – disk usage remains minimal.



Hold:

- CPU: 0.82% – unchanged, system stable.
- Memory: 64.16 MiB / 7.6 GiB (0.82%) – stable.
- Network I/O: 94.7 MB / 127 MB – stable, no spikes.
- Block I/O: 3.71 MB / 0B – minimal.
- PIDs: 7 – stable.



Conclusion:

The test ran from 11:46 to 12:06.

Some requests failed during the test. The main errors were:

- `java.net.BindException: Address already in use` – the system could not open new connections.
- `404 Not Found` – some requests were made to missing endpoints.

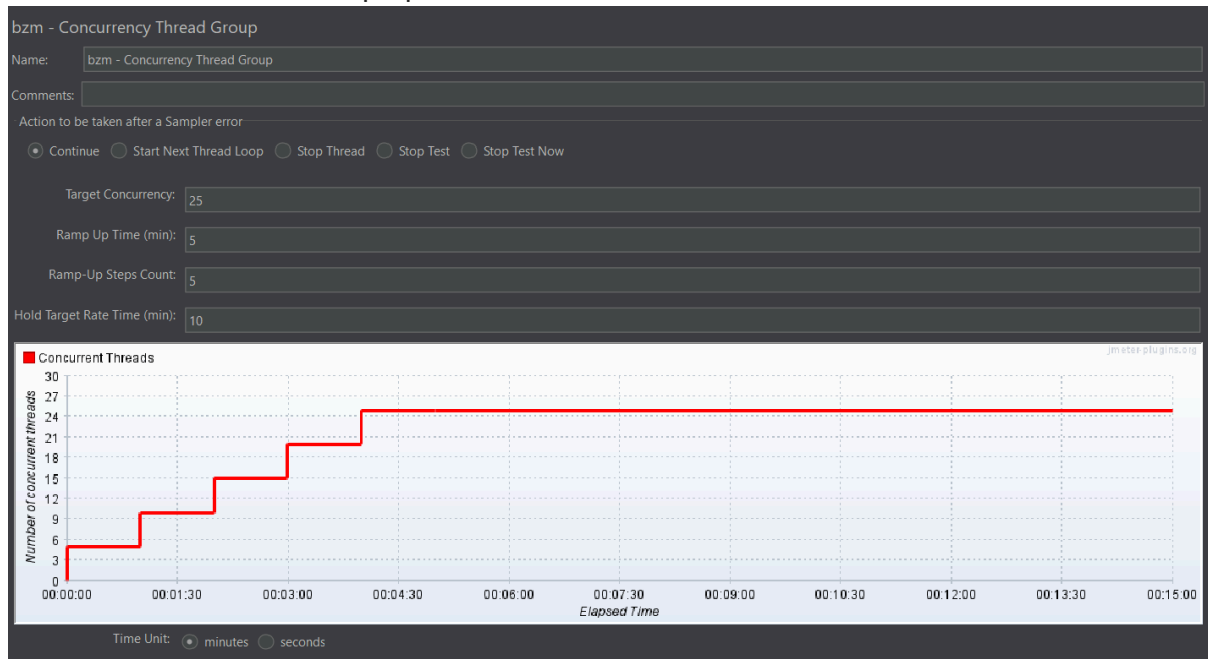
Overall, the application handled most of the load, but it showed stability problems under high load. The system works under normal conditions, but fixes are needed to reduce errors during stress.

The high error rate affected all operations, but `GetCharacters` was the most impacted with an error rate of 38.24%. Other actions like `CreateCharacter` and `UpdateCharacter` failed at a rate of approximately 34%.

The container shows **very low CPU and memory usage** throughout the test. Network traffic increases during peak load but remains manageable. The system stabilizes quickly after the peak. Most errors observed in the load test (`java.net.BindException` and `404 Not Found`) are **not caused by resource limits**, but likely due to connection handling or application configuration. Overall, the container handles the load well.

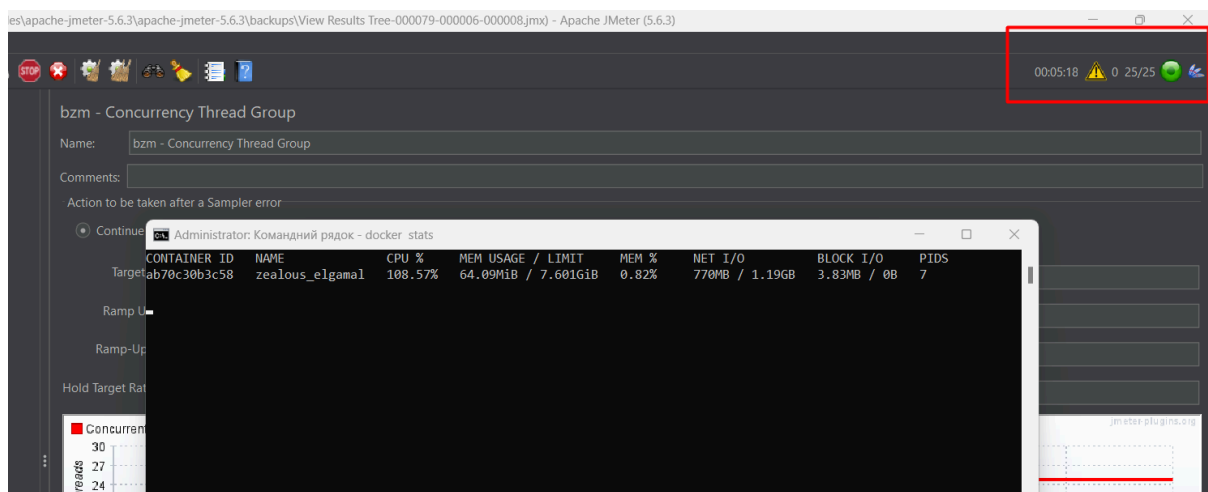
Load Test Configuration

Load testing was performed using a bzm Concurrency Thread Group with a target of 25 concurrent users and a ramp-up of 5 minutes.



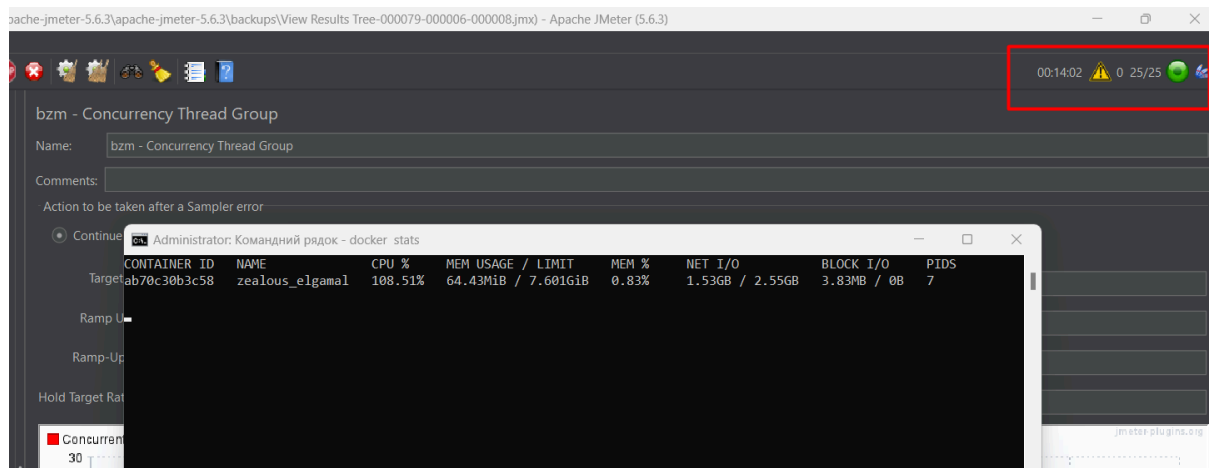
Peak Load:

- CPU: 108.57% – heavily loaded, likely using multiple cores/threads.
- Memory: 64.09 MiB / 7.6 GiB (0.82%) – very low, plenty of memory available.
- Network I/O: 770 MB / 1.19 GB – high network activity.
- Block I/O: 3.83 MB / 0B – minimal disk usage.
- PIDs: 7 – stable.



Hold:

- CPU: 108.51% – CPU load remains high, container continues processing requests.
- Memory: 64.43 MiB / 7.6 GiB (0.83%) – still very low.
- Network I/O: 1.53 GB / 2.55 GB – network traffic continues to grow.
- Block I/O: 3.83 MB / 0B – minimal disk usage.
- PIDs: 7 – stable.



Conclusion:

The test ran from from 12:33 to 12:48.

Overall, the container performed reliably under peak load conditions. CPU usage was high, memory and disk were hardly used, and network traffic increased under load. Overall, the container performed very well under peak conditions.

Optimization:

Since I am using **Docker**, here are some easy ways to make the system faster:

For Docker:

- **Limits:** Give the container more CPU and RAM.
- **Logs:** Don't print too many logs to the screen. This makes the container slow.

For JMeter:

- **Memory:** Give JMeter more memory (RAM) in the settings so it doesn't crash during the Stress test.
- **No GUI:** Always run the test in the console (command line), not the window. This saves a lot of resources.
- **HTTP Keep-Alive:** Haven't configured JMeter to use persistent connections.